

Retail Analytics Platform

End-to-End Data Analytics and Machine Learning Project

Project Documentation

Prepared for portfolio, GitHub README, and interview explanation

Table of Contents

1. 1. Project Overview
2. 2. Problem Statement
3. 3. Technologies Used
4. 4. Project Architecture
5. 5. Folder Structure
6. 6. Phase 1 - Local Data Analytics using Python
7. 7. Data Cleaning
8. 8. Exploratory Data Analysis
9. 9. Charts and Visualizations
10. 10. Phase 2 - SQLite Database Integration
11. 11. SQL Reports Generated
12. 12. Phase 3 - Machine Learning
13. 13. Phase 4 - PySpark Analytics
14. 14. Phase 5 - Docker Setup
15. 15. Phase 6 - HDFS Integration
16. 16. Phase 7 - Spark on Docker
17. 17. Phase 8 - Hive Integration
18. 18. Phase 9 - Airflow Orchestration
19. 19. Main Project Execution
20. 20. Requirements and Virtual Environment
21. 21. Output Files and Reports
22. 22. Git and GitHub Workflow
23. 23. Remove File or Folder from GitHub Only
24. 24. .gitignore File
25. 25. Common Errors and Fixes
26. 26. Business Insights Generated
27. 27. Skills Demonstrated
28. 28. Resume Description and Bullet Points
29. 29. Interview Explanation
30. 30. How to Run the Project
31. 31. Future Enhancements
32. 32. Conclusion

1. Project Overview

Project Title: Retail Analytics Platform - End-to-End Data Analytics and Machine Learning

The objective of this project is to build a complete retail analytics platform that can process retail sales data, clean and transform it, generate business reports, perform exploratory data analysis, train machine learning models, process large-scale data using PySpark, store and query data using SQLite and Hive, and orchestrate pipelines using Apache Airflow.

This project demonstrates both Data Analytics and Data Engineering skills. It is suitable for GitHub, resume projects, interviews, and portfolio presentation.

2. Problem Statement

Retail businesses generate large amounts of sales data from customers, products, orders, regions, categories, and shipping operations. Raw data is usually not directly useful because it may contain missing values, inconsistent formats, duplicate records, unprocessed date columns, and unstructured business metrics.

This project solves that problem by creating a complete analytics pipeline that performs the following operations:

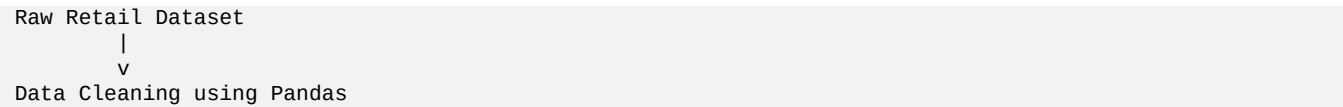
- Clean raw retail sales data.
- Perform exploratory data analysis.
- Generate business reports.
- Store data in SQLite for SQL reporting.
- Process data using PySpark for scalable analytics.
- Use HDFS and Hive through Docker to simulate a big data ecosystem.
- Create Airflow DAGs for workflow orchestration.
- Build machine learning models for prediction.
- Produce final reports for business decision-making.

3. Technologies Used

Area	Tools / Details
Programming Language	Python
Data Analysis Libraries	Pandas, NumPy, Matplotlib, Seaborn
Machine Learning Libraries	Scikit-learn, Joblib
Database	SQLite
Big Data Tools	Apache Spark, PySpark, Hadoop HDFS, Apache Hive
Workflow Orchestration	Apache Airflow
Containerization	Docker, Docker Compose
Version Control	Git, GitHub

4. Project Architecture

The project follows this end-to-end architecture:



```
|
v
EDA and Business Analysis
|
v
SQLite Database Storage
|
v
SQL Business Reports
|
v
PySpark Large-Scale Processing
|
v
HDFS Storage using Docker
|
v
Hive Table Creation and Querying
|
v
Airflow DAG Orchestration
|
v
Final Reports + Charts + ML Models
```

5. Folder Structure

```
retail-analytics-platform/
|
|-- data/
|   |-- train.csv
|
|-- database/
|   |-- retail.db
|
|-- docker/
|   |-- docker-compose.yml
|   |-- airflow/
|   |-- hive/
|
|-- output/
|   |-- charts/
|   |-- reports/
|   |-- spark_reports/
|   |-- hive_reports/
|   |-- models/
|
|-- scripts/
|
|-- spark/
|   |-- pyspark_analysis.py
|
|-- src/
|   |-- data_cleaning.py
|   |-- eda.py
|   |-- model_training.py
|   |-- prediction.py
|
|-- main.py
|-- requirements.txt
|-- README.md
|-- .gitignore
```

6. Phase 1 - Local Data Analytics using Python

6.1 Data Loading

The dataset is loaded using Pandas. The first step is to inspect the structure, shape, data types, missing values, and sample rows.

```
import pandas as pd

df = pd.read_csv("data/train.csv")
print(df.head())
print(df.shape)
print(df.info())
```

6.2 Data Understanding

Important checks performed during data understanding:

```
df.head()
df.tail()
df.shape
df.info()
df.describe()
df.isnull().sum()
df.duplicated().sum()
```

These commands help identify row count, column count, data types, null values, duplicate records, and basic numerical summaries.

7. Data Cleaning

The data cleaning process prepares the raw dataset for analytics, reporting, and machine learning.

- Convert date columns into datetime format.
- Handle missing values.
- Remove duplicate records.
- Create new time-based and shipping-related columns.
- Prepare clean data for reports and models.

```
df["Order Date"] = pd.to_datetime(df["Order Date"])
df["Ship Date"] = pd.to_datetime(df["Ship Date"])

df["Year"] = df["Order Date"].dt.year
df["Month"] = df["Order Date"].dt.month
df["Quarter"] = df["Order Date"].dt.quarter
df["Ship_Days"] = (df["Ship Date"] - df["Order Date"]).dt.days
```

New Columns Created

- Year
- Month
- Quarter
- Ship_Days

These columns help in time-based analysis, quarterly analysis, monthly sales trend analysis, and shipping performance analysis.

8. Exploratory Data Analysis

EDA was performed to understand business performance and identify important trends in the retail data.

- Total Sales
- Total Profit
- Sales by Category
- Sales by Region
- Sales by Segment
- Sales by State
- Sales by Sub-Category
- Monthly Sales Trend
- Top Products
- Top Customers
- Profit Analysis
- Shipping Days Analysis

```
category_sales = df.groupby("Category")["Sales"].sum().reset_index()
print(category_sales)
```

9. Charts and Visualizations

Charts were created using Matplotlib and saved inside the output/charts folder.

- Sales by Category
- Sales by Region
- Sales by Segment
- Monthly Sales Trend
- Profit by Sub-Category
- Top 10 Products

```
import matplotlib.pyplot as plt

category_sales.plot(kind="bar", x="Category", y="Sales")
plt.title("Sales by Category")
plt.xlabel("Category")
plt.ylabel("Sales")
plt.tight_layout()
plt.savefig("output/charts/sales_by_category.png")
plt.show()
```

10. Phase 2 - SQLite Database Integration

SQLite was used to store the cleaned retail data in a database. This makes it possible to run SQL queries and generate reports in a structured way.

Item	Value
Database file	database/retail.db
Table name	retail_sales

```
import sqlite3

conn = sqlite3.connect("database/retail.db")
df.to_sql("retail_sales", conn, if_exists="replace", index=False)
```

```
conn.close()
```

11. SQL Reports Generated

Business reports were generated using SQL queries and saved inside output/reports.

- category_report.csv
- region_report.csv
- segment_report.csv
- state_report.csv
- sub_category_report.csv
- monthly_sales_report.csv
- top_10_products.csv

```
SELECT Category, SUM(Sales) AS Total_Sales
FROM retail_sales
GROUP BY Category
ORDER BY Total_Sales DESC;
```

12. Phase 3 - Machine Learning

Machine learning models were trained to perform prediction based on retail data. This phase shows practical ML workflow knowledge including feature selection, train-test split, model training, evaluation, and saving models.

ML Tasks

- Regression model for sales or profit prediction.
- Classification model for business category or status prediction.
- Model comparison.
- Model saving using Joblib.

Algorithms Used

Model Type	Algorithms
Regression	Linear Regression, Decision Tree Regressor, Random Forest Regressor
Classification	Logistic Regression, Decision Tree Classifier, Random Forest Classifier, KNN, Gaussian Naive Bayes, SVC

Model Training Flow

```
Load cleaned data
|
Feature selection
|
Encoding categorical columns
|
Train-test split
|
Model training
|
Model evaluation
|
Save model
```

```
from sklearn.model_selection import train_test_split
```

```

from sklearn.ensemble import RandomForestRegressor
import joblib

X = df[["Quantity", "Discount", "Profit"]]
y = df["Sales"]

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

model = RandomForestRegressor()
model.fit(X_train, y_train)

joblib.dump(model, "output/models/sales_model.pkl")

```

13. Phase 4 - PySpark Analytics

PySpark was used to process data in a distributed way. This phase is important for demonstrating big data analytics skills.

Item	Value
PySpark file	spark/pyspark_analysis.py
Output folder	output/spark_reports/

PySpark Reports Created

- category_summary
- monthly_summary
- region_summary
- segment_summary
- state_summary
- sub_category_summary
- top_10_customers
- top_10_products
- top_product_by_category

```

from pyspark.sql import SparkSession
from pyspark.sql.functions import sum, count

spark = SparkSession.builder      .appName("Retail Analytics Platform")      .getOrCreate()

df = spark.read.csv("data/train.csv", header=True, inferSchema=True)

category_summary = df.groupBy("Category").agg(
    sum("Sales").alias("Total_Sales"),
    sum("Profit").alias("Total_Profit"),
    count("*").alias("Total_Orders")
)

category_summary.show()

```

Running PySpark Locally

```

python3 spark/pyspark_analysis.py
spark-submit spark/pyspark_analysis.py

```

14. Phase 5 - Docker Setup

Docker was used to run services like Hadoop, Spark, Hive, Airflow, and PostgreSQL in containers.

Service	Purpose
Hadoop NameNode	Stores HDFS metadata.
Hadoop DataNode	Stores HDFS data blocks.
Spark Master	Coordinates Spark jobs.
Spark Worker	Executes Spark tasks.
Hive Server	Runs Hive SQL queries.
Airflow Webserver	Provides Airflow UI.
Airflow Scheduler	Schedules and runs DAG tasks.
PostgreSQL	Airflow metadata database.

Docker Commands

```
cd docker
docker compose up -d
docker ps
docker compose down
```

15. Phase 6 - HDFS Integration

HDFS was used to store the retail dataset in distributed storage. This simulates a real big data platform.

```
docker exec -it retail-namenode hdfs dfs -mkdir -p /retail_analytics/input
docker exec -it retail-namenode hdfs dfs -put data/train.csv /retail_analytics/input/
docker exec -it retail-namenode hdfs dfs -ls /retail_analytics/input
```

Path Type	HDFS Path
Input path	/retail_analytics/input/train.csv
Output path	/retail_analytics/output/

16. Phase 7 - Spark on Docker

Spark was executed inside Docker to process data from HDFS and generate reports.

```
docker exec -it spark-master spark-submit --master spark://spark-master:7077
/opt/spark/jobs/pyspark_analysis.py
```

Service UI	URL
Spark Master UI	http://localhost:8080
Spark Worker UI	http://localhost:8081

17. Phase 8 - Hive Integration

Hive was used to create tables and run SQL queries on the big data stored in HDFS.

```
CREATE DATABASE IF NOT EXISTS retail_db;
USE retail_db;
```

```
CREATE TABLE retail_sales_raw (
  row_id INT,
  order_id STRING,
  order_date STRING,
  ship_date STRING,
  ship_mode STRING,
  customer_id STRING,
  customer_name STRING,
  segment STRING,
  country STRING,
  city STRING,
  state STRING,
  postal_code STRING,
  region STRING,
  product_id STRING,
  category STRING,
  sub_category STRING,
  product_name STRING,
  sales DOUBLE
)
ROW FORMAT DELIMITED
FIELDS TERMINATED BY ','
STORED AS TEXTFILE;
```

```
LOAD DATA INPATH '/retail_analytics/input/train.csv'
INTO TABLE retail_sales_raw;
```

```
SELECT category, SUM(sales) AS total_sales
FROM retail_sales_raw
GROUP BY category
ORDER BY total_sales DESC;
```

Hive query outputs are saved in output/hive_reports/.

18. Phase 9 - Airflow Orchestration

Apache Airflow was used to automate the retail data pipeline. It controls tasks such as validation, cleaning, analytics script execution, PySpark job execution, and report generation.

- Airflow Webserver
- Airflow Scheduler
- Airflow DAG
- PythonOperator / BashOperator

Airflow DAG Purpose

- Validate input data.
- Run data cleaning.
- Run analytics scripts.
- Run PySpark job.
- Generate reports.

```
# Airflow UI
http://localhost:8085
```

```
# Check DAG list
docker exec -it retail-airflow-webserver airflow dags list
```

```
# Trigger DAG manually
docker exec -it retail-airflow-webserver airflow dags trigger retail_basic_validation_pipeline

# Check DAG runs
docker exec -it retail-airflow-webserver airflow dags list-runs -d retail_basic_validation_pipeline
```

19. Main Project Execution

The main file controls the local workflow and runs the local pipeline.

File	Purpose
main.py	Runs data cleaning, EDA, SQLite reporting, model training, and prediction workflow.

```
python3 main.py
```

20. Requirements and Virtual Environment

The project dependencies are stored in requirements.txt.

```
pandas
numpy
matplotlib
seaborn
scikit-learn
joblib
pyspark
apache-airflow
```

Virtual Environment Setup

```
python3 -m venv reatil_sales
source reatil_sales/bin/activate
pip install -r requirements.txt
```

21. Output Files and Reports

Output Type	Location
Charts	output/charts/
SQL Reports	output/reports/
PySpark Reports	output/spark_reports/
Hive Reports	output/hive_reports/
ML Models	output/models/
SQLite Database	database/retail.db

22. Git and GitHub Workflow

Git and GitHub were used for version control and project publishing.

```
git init
git status
git add .
git commit -m "Completed end-to-end retail analytics platform"
```

Add or Fix Remote Repository

```
git remote add origin <your_github_repo_url>

git remote set-url origin <correct_github_repo_url>

git remote -v
```

Push to GitHub

```
git branch -M main
git push -u origin main
```

23. Remove File or Folder from GitHub Only

Sometimes a file or folder should be removed from GitHub but kept on the local machine. In that case, use `--cached`.

```
# Remove file from GitHub only
git rm --cached "filename"
git commit -m "Remove file from repository"
git push origin main

# Remove folder from GitHub only
git rm -r --cached "foldername"
git commit -m "Remove folder from repository"
git push origin main
```

Add to .gitignore

```
echo "foldername/" >> .gitignore
git add .gitignore
git commit -m "Ignore unwanted folder"
git push origin main
```

Check Tracked Files

```
git ls-files
git ls-files | grep -i filename
```

24. .gitignore File

The `.gitignore` file prevents unnecessary files from being uploaded to GitHub.

```
# Python
__pycache__/
*.pyc
*.pyo
*.pyd

# Virtual Environment
venv/
```

```
.venv/  
reatil_sales/  
  
# Output files  
output/  
*.csv  
*.xlsx  
  
# Database  
*.db  
  
# Logs  
logs/  
*.log  
  
# Docker/Airflow unwanted files  
airflow/logs/  
airflow/plugins/  
airflow/__pycache__/  
  
# Mac files  
.DS_Store
```

25. Common Errors and Fixes

Error 1 - Wrong GitHub Remote

```
# Problem: origin points to wrong repository  
  
# Fix:  
git remote set-url origin correct_repo_url
```

Error 2 - Pathspec Did Not Match Any Files

```
fatal: pathspec 'foldername' did not match any files
```

Reason: The file or folder is not tracked by Git, or the name/path is wrong.

```
git ls-files  
git ls-files | grep -i foldername
```

Error 3 - git ls is Not a Git Command

```
# Wrong  
git ls  
  
# Correct  
git ls-files
```

Error 4 - Repository Is Empty

```
fatal: couldn't find remote ref HEAD  
  
# Fix:  
git branch -M main  
git push -u origin main
```

Error 5 - Docker Container Not Running

```
docker ps
```

```
docker compose up -d
```

Error 6 - Airflow DAG Not Visible

```
docker exec -it retail-airflow-webserver airflow dags list
docker restart retail-airflow-webserver retail-airflow-scheduler
```

26. Business Insights Generated

This project helps answer important business questions such as:

- Which category generates the highest sales?
- Which region performs best?
- Which segment contributes most revenue?
- Which products are top-selling?
- Which states generate high sales?
- What is the monthly sales trend?
- Which sub-categories are profitable?
- How many days does shipping take?
- What are the most important sales KPIs?

27. Skills Demonstrated

Skill Area	Skills Demonstrated
Data Analytics	Data cleaning, EDA, KPI generation, reports, visualization
Data Engineering	ETL pipeline, SQLite, PySpark, HDFS, Hive, Docker
Machine Learning	Feature engineering, regression, classification, model evaluation, model saving
DevOps and Tools	Git, GitHub, Docker, Docker Compose, Airflow

28. Resume Description and Bullet Points

Resume Description

Retail Analytics Platform - End-to-End Data Analytics and Machine Learning

Built an end-to-end retail analytics platform using Python, Pandas, SQLite, PySpark, Hive, HDFS, Docker, and Airflow. Designed data cleaning workflows, generated business reports, created visual dashboards, trained machine learning models, processed retail data using Spark, stored data in HDFS, queried data using Hive, and automated pipeline tasks using Airflow.

Resume Bullet Points

- Built an end-to-end retail analytics and machine learning platform using Python, Pandas, SQLite, PySpark, Hive, HDFS, Docker, and Airflow.
- Cleaned and transformed retail sales data by handling missing values, converting date columns, and creating time-based features.
- Generated business reports for category, region, segment, state, sub-category, monthly sales, and top products using SQL and PySpark.
- Integrated SQLite database for structured storage and SQL-based reporting.

- Implemented PySpark jobs for distributed data processing and large-scale analytics.
- Used HDFS and Hive inside Docker to simulate a big data ecosystem.
- Created Airflow DAGs to orchestrate validation and analytics workflows.
- Trained machine learning models for retail prediction tasks and saved models using Joblib.
- Managed source code using Git and GitHub with proper project structure and documentation.

29. Interview Explanation

This project is an end-to-end retail analytics platform. I started with raw retail sales data and cleaned it using Python and Pandas. After cleaning, I performed exploratory data analysis to understand sales, profit, category performance, regional performance, monthly trends, and top products.

Then I stored the cleaned data in SQLite and generated SQL-based business reports. After that, I used PySpark to process the same data in a distributed way and generated large-scale analytical reports.

To simulate a big data environment, I used Docker containers for Hadoop HDFS, Spark, Hive, and Airflow. I uploaded the dataset to HDFS, processed it using Spark, created Hive tables, and queried the data using Hive SQL. Finally, I used Airflow to orchestrate pipeline tasks.

The project also includes machine learning models for prediction and a complete GitHub workflow with proper folder structure, .gitignore, and documentation.

30. How to Run the Project

```
# 1. Clone repository
git clone <your_github_repo_url>

# 2. Enter project folder
cd retail-analytics-platform

# 3. Create virtual environment
python3 -m venv reatil_sales

# 4. Activate environment
source reatil_sales/bin/activate

# 5. Install requirements
pip install -r requirements.txt

# 6. Run main pipeline
python3 main.py

# 7. Run PySpark analysis
python3 spark/pyspark_analysis.py

# 8. Start Docker services
cd docker
docker compose up -d

# 9. Check containers
docker ps
```

Service	URL
Spark Master UI	http://localhost:8080
Spark Worker UI	http://localhost:8081
Airflow UI	http://localhost:8085

31. Future Enhancements

- Tableau or Power BI dashboard.
- Streamlit web dashboard.
- Real-time Kafka data streaming.
- Cloud deployment on AWS or GCP.
- Automated CI/CD pipeline.
- Advanced ML model tuning.
- Data quality monitoring.
- API-based prediction service using FastAPI.

32. Conclusion

This project successfully demonstrates a complete data analytics and data engineering workflow. It starts from raw retail data and moves through cleaning, analysis, reporting, database storage, machine learning, big data processing, HDFS storage, Hive querying, Docker containerization, Airflow orchestration, and GitHub deployment.

It is suitable for showcasing skills in Data Analytics, Data Engineering, Big Data, Machine Learning, Python, SQL, PySpark, Hive, HDFS, Docker, Airflow, and GitHub.